

浙江大学



计算机组成与设计实验报告

项目名称: cache 仿真器与性能分析

姓 名: 李维扬

学 号: 3230101883

学 院: 信息与工程学院

专 业: 电子科学与技术

指导老师: 赵武锋

2026 年 1 月 7 日

目 录

一	实验概述	2
二	任务一：缓存模拟器实现	2
	2.1 核心数据结构	2
	2.2 cache 初始化	3
	2.3 perform_access 函数实现	4
	2.4 flush 函数实现	6
三	实验测试与验证	7
	3.1 测试配置	7
	3.2 测试结果	7
	3.3 验证	8
四	任务二：性能评估	9
	4.1 实验 2-1：工作集特征	9
	4.2 实验 2-2：块大小影响	11
	4.3 实验 2-3：相联度影响	12
	4.4 实验 2-4：内存带宽分析	14

一 实验概述

本实验旨在实现一个跟踪驱动的缓存模拟器，用于评估不同缓存架构特性对性能的影响。模拟器通过读取内存访问跟踪文件，模拟数据在缓存中的进出行为，收集命中率、缺失率、内存流量等关键性能指标。实验过程中需要实现缓存初始化、访问处理和刷新三个核心功能，并验证模拟器在不同配置下的正确性。

二 任务一：缓存模拟器实现

高速缓存模拟器将可以根据命令行中给出的参数进行配置，并且必须支持以下功能：

- Total cache size
- Block size
- Unified vs. split I- and D-caches
- Associativity
- Write back vs. write through
- Write allocate vs. write no allocate

模拟器必须跟踪：

- Number of instruction references
- Number of data references
- Number of instruction misses
- Number of data misses
- Number of words fetched from memory
- Number of words copied back to memory

2.1 核心数据结构

缓存模拟器采用双向链表实现 LRU 替换策略。每个缓存行包含标签(tag)和脏位(dirty)字段，通过 LRU_next 和 LRU_prev 指针维护访问顺序。缓存结构体包含大小、关联度、组数、索引掩码等配置信息，以及指向每组 LRU 链表头尾的指针数组。统计结构体记录访问次数、缺失次数、替换次数、需求取回字数和写回字数。

```
1 typedef struct cache_line_ {
2     unsigned tag;
3     int dirty;
4     struct cache_line_ *LRU_next;
5     struct cache_line_ *LRU_prev;
6 } cache_line, *Pcache_line;
7
8 typedef struct cache_ {
9     int size;                /* cache size */
```

```

10  int associativity;      /* cache associativity */
11  int n_sets;            /* number of cache sets */
12  unsigned index_mask;   /* mask to find cache index */
13  int index_mask_offset; /* number of zero bits in mask */
14  Pcache_line *LRU_head; /* head of LRU list for each set */
15  Pcache_line *LRU_tail; /* tail of LRU list for each set */
16  int *set_contents;     /* number of valid entries in set */
17 } cache, *Pcache;

```

cache_line 结构体表示单个缓存行。其中 tag 用于地址匹配，dirty 标记该行是否被修改过（用于写回策略），双向链表指针用于 LRU 排序。cache 结构体描述整个缓存的配置，index_mask 和 index_mask_offset 用于从地址中快速提取组索引，LRU_head 和 LRU_tail 数组分别指向每组的 LRU 链表头尾。

2.2 cache 初始化

先实现 cache 的初始化，命令行参数获取和设置代码框架已经给出，只需要在 cache.c 中实现 init_cache 函数即可。

首先将指令和数据的统计计数器清零，然后根据 cache_split 标志判断是统一缓存还是分离缓存模式。对于统一缓存，仅初始化 c1 结构体并使 icache 和 dcache 同时指向它；对于分离缓存，分别初始化 c1 作为指令缓存、c2 作为数据缓存。

初始化过程包括计算组数、设置索引掩码偏移量为块大小的对数、构造索引掩码用于地址解析，并为每组分配 LRU 头尾指针和内容计数器。

```

1  void init_cache() {
2      int i;
3      /* Initialize statistics */
4      cache_stat_inst.accesses = 0;
5      cache_stat_inst.misses = 0;
6      cache_stat_inst.replacements = 0;
7      cache_stat_inst.demand_fetches = 0;
8      cache_stat_inst.copies_back = 0;
9      // ... data stats similarly
10
11     if (cache_split) {
12         /* Initialize instruction cache */
13         c1.size = cache_isize;
14         c1.associativity = cache_assoc;
15         c1.n_sets = cache_isize / (cache_block_size * cache_assoc);
16         c1.index_mask_offset = LOG2(cache_block_size);
17         c1.index_mask = (c1.n_sets - 1) << c1.index_mask_offset;
18
19         c1.LRU_head = (Pcache_line *)calloc(c1.n_sets,
20 sizeof(Pcache_line));
21         c1.LRU_tail = (Pcache_line *)calloc(c1.n_sets,
22 sizeof(Pcache_line));
23         c1.set_contents = (int *)calloc(c1.n_sets, sizeof(int));
24         icache = &c1;

```

```

23     // ... similarly for c2 as dcache
24 } else {
25     /* Unified cache: icache and dcache point to same structure */
26     c1.n_sets = cache_usize / (cache_block_size * cache_assoc);
27     // ... initialize c1
28     icache = &c1;
29     dcache = &c1;
30 }
31 }

```

组数计算公式为

$$n_{\text{sets}} = \frac{\text{cache size}}{\text{block size} \times \text{associativity}}$$

索引掩码的构造通过将 $(n_{\text{sets}} - 1)$ 左移 `index_mask_offset` 位实现，其中偏移量等于块大小的对数。对于 8KB 缓存、16 字节块、直接映射的配置，组数为 512，索引掩码为 `0x1FF0`（9 位索引左移 4 位）。统一缓存模式下 `icache` 和 `dcache` 指向同一结构体，实现指令和数据共享缓存空间。

2.3 perform_access 函数实现

该函数是缓存模拟的核心。我们首先实现基础的统一缓存直接映射功能，然后逐步添加可变关联度、写策略和指令数据分离统计等功能。

2.3.1 基础功能

首先根据访问类型确定使用指令缓存还是数据缓存，以及对应的统计结构体。地址解析通过索引掩码提取组索引，通过移位操作提取标签。32 位地址被划分为三部分：低位的块内偏移、中间的组索引、高位的标签。

```

1 void perform_access(addr, access_type)
2     unsigned addr, access_type;
3 {
4     unsigned index, tag;
5     Pcache c;
6     Pcache_stat stat;
7
8     /* Select cache and stats based on access type */
9     if (access_type == TRACE_INST_LOAD) {
10         c = icache; stat = &cache_stat_inst; is_write = FALSE;
11     } else {
12         c = dcache; stat = &cache_stat_data;
13         is_write = (access_type == TRACE_DATA_STORE);
14     }
15     stat->accesses++;
16
17     /* Calculate index and tag from address */
18     index = (addr & c->index_mask) >> c->index_mask_offset;
19     tag = addr >> (c->index_mask_offset + LOG2(c->n_sets));

```

通过 `index_mask` 与地址按位与后右移可得组索引, 将地址右移(`index_mask_offset + LOG2(n_sets)`)位可得标签。若缓存组为空, 说明未命中, 需要动态分配一个 cache 块并更新统计信息; 否则遍历链表查找匹配的标签。

2.3.2 可变关联度与 LRU 替换

这一步骤基于双向链表实现组相联缓存的 LRU 替换策略。

缓存查找遍历目标组的 LRU 链表, 若标签匹配则命中, 将该行移至链表头部表示最近使用。每个组通过 `set_contents` 记录有效条目数, 当组已满时选择链表尾部 (即最久未使用) 的块进行替换。

```

1  /* Search for hit in the cache set */
2  line = c->LRU_head[index];
3  while (line != NULL) {
4      if (line->tag == tag) {
5          /* Cache hit: update LRU order */
6          delete(&c->LRU_head[index], &c->LRU_tail[index], line);
7          insert(&c->LRU_head[index], &c->LRU_tail[index], line);
8          if (is_write) {
9              line->dirty = TRUE;
10             if (!cache_writeback) stat->copies_back++; /* Write through */
11         }
12         return;
13     }
14     line = line->LRU_next;
15 }

```

命中时调用 `delete` 和 `insert` 将该行移至链表头部。

2.3.3 写策略与 dirty 位管理

这一步骤实现写回(WB)、写直通(WT)和写分配(WA)、非写分配(WNA)策略。写命中时设置 `dirty` 标志位, 写直通模式下同时累加写回流量。写缺失时, 非写分配模式直接写入内存而不分配缓存行; 写分配模式则先取入数据块再写入。替换时, 若被驱逐块的 `dirty` 位为 1 且采用写回策略, 则需将数据写回内存并累加写回流量。

```

1  /* Cache miss */
2  stat->misses++;
3
4  /* Write miss with no-write-allocate: write directly to memory */
5  if (is_write && !cache_writealloc) {
6      stat->copies_back++;
7      return;
8  }
9
10 /* Fetch block from memory */
11 stat->demand_fetches += words_per_block;
12
13 /* Evict LRU line if set is full */
14 if (c->set_contents[index] >= c->associativity) {

```

```

15     stat->replacements++;
16     line = c->LRU_tail[index];
17     if (line->dirty && cache_writeback)
18         stat->copies_back += words_per_block; /* Write back dirty block
19 */
19     delete(&c->LRU_head[index], &c->LRU_tail[index], line);
20     new_line = line; /* Reuse evicted line structure */
21 } else {
22     new_line = (Pcache_line)malloc(sizeof(cache_line));
23 }
24
25 /* Initialize and insert new line */
26 new_line->tag = tag;
27 new_line->dirty = is_write;
28 insert(&c->LRU_head[index], &c->LRU_tail[index], new_line);
29 c->set_contents[index]++;
30 }

```

缺失处理体现了写策略的差异。写回模式下脏块在被驱逐时才写回内存（每次写回 words_per_block 字），写直通模式下每次写操作都立即写回。新分配或复用的缓存行根据访问类型设置 dirty 标志，数据写时置为 TRUE，读操作时置为 FALSE。

2.4 flush 函数实现

该函数在模拟结束时遍历所有缓存行，将脏块写回内存。对于分离缓存需分别遍历指令缓存和数据缓存，统一缓存则仅遍历 dcache（与 icache 指向同一结构）。遍历过程中检查每行的脏位，若为脏且采用写回策略，则累加对应统计结构体的写回字数，并清除脏位。

```

1 void flush() {
2     int i;
3     Pcache_line line;
4
5     /* Flush instruction cache (only if split cache) */
6     if (cache_split) {
7         for (i = 0; i < icache->n_sets; i++) {
8             line = icache->LRU_head[i];
9             while (line != NULL) {
10                 if (line->dirty && cache_writeback) {
11                     cache_stat_inst.copies_back += words_per_block;
12                     line->dirty = FALSE;
13                 }
14                 line = line->LRU_next;
15             }
16         }
17     }
18
19     /* Flush data cache (or unified cache) */
20     for (i = 0; i < dcache->n_sets; i++) {
21         line = dcache->LRU_head[i];
22         while (line != NULL) {
23             if (line->dirty && cache_writeback) {

```

```

24         cache_stat_data.copies_back += words_per_block;
25         line->dirty = FALSE;
26     }
27     line = line->LRU_next;
28 }
29 }
30 }

```

flush 函数确保模拟结束时所有脏数据都被统计到写回流量中。这对于写回策略尤为重要，因为脏块可能在程序结束时仍驻留在缓存中。统一缓存模式下 icache 和 dcache 指向同一结构，因此仅遍历 dcache 即可覆盖所有缓存行。

三 实验测试与验证

3.1 测试配置

基本测试配置采用 8KB 统一缓存、16 字节块大小、直接映射（关联度 1）、写回策略和写分配策略。使用 Python 脚本自动化运行多组测试，包括公开测试用例、参数变化测试。测试命令如下：

```
1 ./sim -bs 16 -us 8192 -a 1 -wb -wa traces/spice100.trace
```

3.2 测试结果

对于 spice100.trace 的基本配置测试，指令访问 71 次缺失 20 次（缺失率 28.17%），数据访问 29 次缺失 11 次（缺失率 37.93%），总计需求取回 124 字、写回 12 字。测试输出示例：

```

1  *** CACHE SETTINGS ***
2  Unified I- D-cache
3  Size:      8192
4  Associativity:      1
5  Block size:  16
6  Write policy:      WRITE BACK
7  Allocation policy:  WRITE ALLOCATE
8
9  *** CACHE STATISTICS ***
10 INSTRUCTIONS
11 accesses:  71
12 misses:    20
13 miss rate: 0.2817 (hit rate 0.7183)
14 replace:   1
15 DATA
16 accesses:  29
17 misses:    11
18 miss rate: 0.3793 (hit rate 0.6207)
19 replace:   0
20 TRAFFIC (in words)

```



```
21 demand fetch: 124
22 copies back: 12
```

不同块大小测试显示，块大小增加可降低缺失率，但增加传输流量。不同关联度测试表明在该跟踪下关联度变化对性能影响有限。写策略对比显示写直通非写分配模式下数据缺失率略高（44.83%），但总流量可能更低。

3.3 验证

模拟器输出与预期行为一致：命中时正确更新 LRU 顺序，缺失时正确处理替换和写回，统计数据符合缓存工作原理。验证要点包括：需求取回字数等于缺失次数乘以每块字数，写回字数在写回模式下等于脏块驱逐次数乘以每块字数加上 flush 时的脏块数乘以每块字数。

为便于测试和验证，编写了两个 Python 脚本。

run_tests.py 用于自动运行多种配置的测试用例，包括公开测试集、不同块大小测试、不同关联度测试、写策略对比以及分离缓存测试，用于快速验证模拟器的基本功能。

为了直观地验证和展示模拟器输出的正确性，创建 validate.py 用于严格验证模拟器输出与 sim.pdf 中 Table 1 的 22 组预期值是否完全一致，可直观表示模拟器工作正常与否。

CS	I- vs D-	BS	Ass	Write	Alloc	Instructions		Data		Total	
						Misses	Repl	Misses	Repl	DF	CB
8 K	Split	16	1	WB	WA	24681	24173	8283	7818	131856	12024
16 K	Split	16	1	WB	WA	11514	10560	5839	5051	69412	8008
32 K	Split	16	1	WB	WA	5922	4321	1567	520	29956	3628
64 K	Split	16	1	WB	WA	2619	484	1290	103	15636	3324
8 K	Unified	16	1	WB	WA	36136	35787	21261	21098	229588	37844
8 K	Unified	32	1	WB	WA	26673	26502	19561	19476	369872	69104
8 K	Unified	64	1	WB	WA	23104	23029	20377	20324	695696	136112
32 K	Split	128	1	WB	WA	1964	1726	459	280	77536	7296
8 K	Split	64	2	WB	WA	6590	6462	3160	3032	156000	18880
8 K	Split	64	4	WB	WA	6025	5897	875	747	110400	7296
8 K	Split	64	8	WB	WA	6435	6307	803	675	115808	6656
8 K	Split	64	16	WB	WA	6536	6408	799	671	117360	6624
8 K	Split	64	128	WB	WA	6523	6395	790	662	117008	6576
1 K	Split	64	2	WB	WA	44962	44946	24767	24751	1115664	149200
1 K	Split	64	8	WB	WA	45885	45869	22808	22792	1099088	112480
1 K	Split	64	16	WB	WA	45969	45953	20667	20651	1066176	90416
8 K	Split	16	1	WT	WA	24681	24173	8283	7818	131856	66538
8 K	Split	32	1	WT	WA	15868	15612	7504	7264	186976	66538
8 K	Split	64	2	WT	WA	6590	6462	3160	3032	156000	66538
8 K	Split	16	1	WB	WNA	24681	24173	14904	6688	127304	14643
8 K	Split	32	1	WB	WNA	15868	15612	15098	6421	180200	22033
8 K	Split	64	2	WB	WNA	6590	6462	8638	2726	151104	13624

validate.py 运行测试结果如下：

```
PS C:\Users\PC\Desktop\project2> & "D:\Anaconda\python.exe" "c:\Users\PC\Desktop\project2\validate.py"
=====
Cache Simulator Validation - Table 1 (spice.trace)
=====
```

CS	Type	BS	Ass	Wr	Al	I_Miss	I_Repl	D_Miss	D_Repl	DF	CB	Status
8K	Split	16	1	WB	WA	24681	24173	8283	7818	131856	12024	PASS
16K	Split	16	1	WB	WA	11514	10560	5839	5051	69412	8008	PASS
32K	Split	16	1	WB	WA	5922	4321	1567	520	29956	3628	PASS
64K	Split	16	1	WB	WA	2619	484	1290	103	15636	3324	PASS
8K	Unified	16	1	WB	WA	36136	35787	21261	21098	229588	37844	PASS
8K	Unified	32	1	WB	WA	26673	26502	19561	19476	369872	69104	PASS
8K	Unified	64	1	WB	WA	23104	23029	20377	20324	695696	136112	PASS
32K	Split	128	1	WB	WA	1964	1726	459	280	77536	7296	PASS
8K	Split	64	2	WB	WA	6590	6462	3160	3032	156000	18880	PASS
8K	Split	64	4	WB	WA	6025	5897	875	747	110400	7296	PASS
8K	Split	64	8	WB	WA	6435	6307	803	675	115808	6656	PASS
8K	Split	64	16	WB	WA	6536	6408	799	671	117360	6624	PASS
8K	Split	64	128	WB	WA	6523	6395	790	662	117008	6576	PASS
1K	Split	64	2	WB	WA	44962	44946	24767	24751	1115664	149200	PASS
1K	Split	64	8	WB	WA	45885	45869	22808	22792	1099088	112480	PASS
1K	Split	64	16	WB	WA	45969	45953	20667	20651	1066176	90416	PASS
8K	Split	16	1	WT	WA	24681	24173	8283	7818	131856	66538	PASS
8K	Split	32	1	WT	WA	15868	15612	7504	7264	186976	66538	PASS
8K	Split	64	2	WT	WA	6590	6462	3160	3032	156000	66538	PASS
8K	Split	16	1	WB	WNA	24681	24173	14904	6688	127304	14643	PASS
8K	Split	32	1	WB	WNA	15868	15612	15098	6421	180200	22033	PASS
8K	Split	64	2	WB	WNA	6590	6462	8638	2726	151104	13624	PASS

```
=====
Results: 22 passed, 0 failed out of 22 tests
=====
PS C:\Users\PC\Desktop\project2> 
```

验证结果表明所有 22 组测试全部通过，指令缺失次数、指令替换次数、数据缺失次数、数据替换次数、需求取回字数和写回字数均与预期值完全一致。

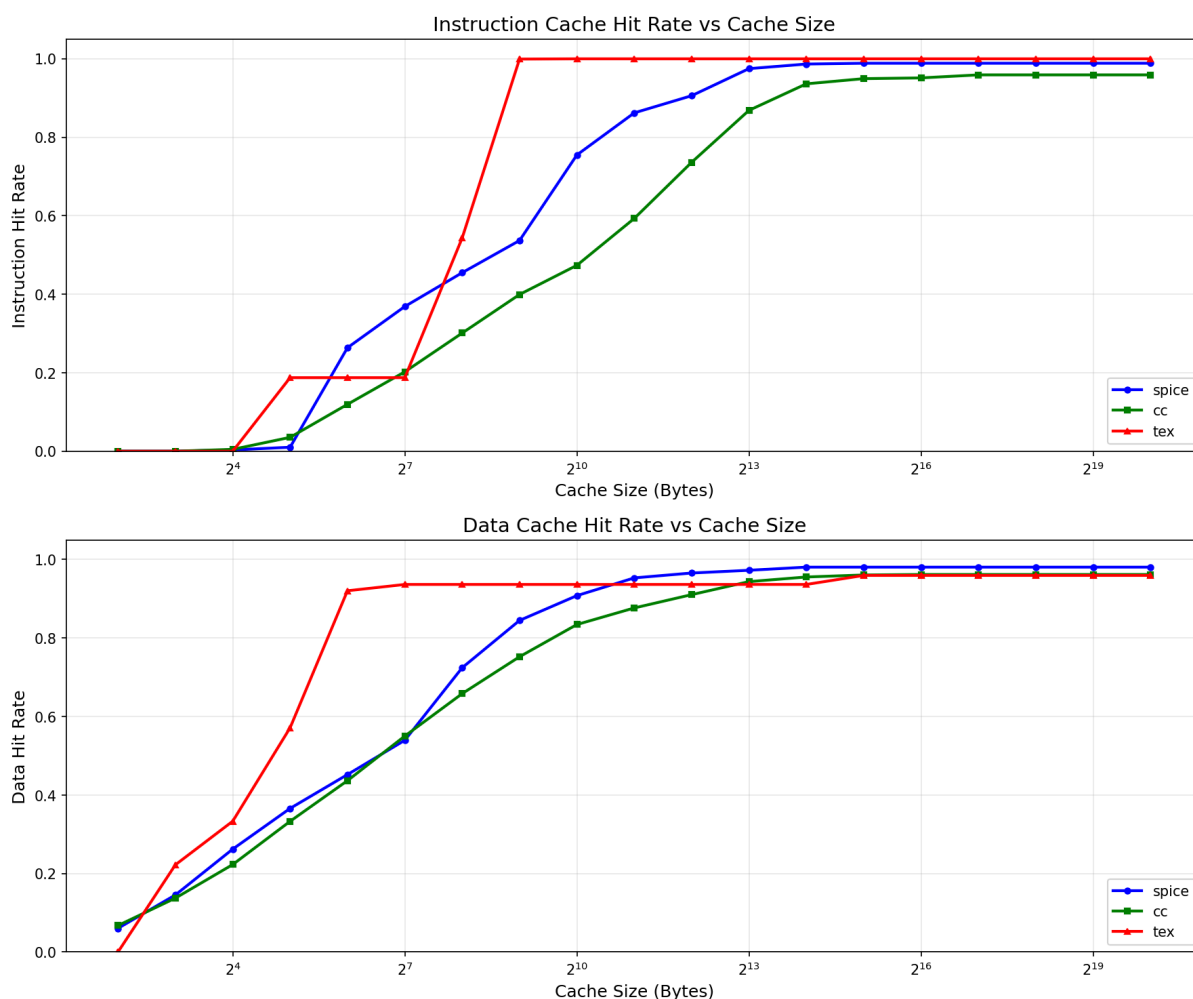
四 任务二：性能评估

- characterize the working set size of the three sample traces given.
- evaluate the Impact of Block Size on performance
- evaluate the Impact of Associativity on performance
- evaluate the Impact of Memory Bandwidth on performance

4.1 实验 2-1：工作集特征

本实验旨在确定程序的指令和数据工作集大小。采用分离式缓存、全相联配置、4 字节块大小、写回+写分配策略。通过逐步增大缓存容量，观察命中率变化趋势。当命中率趋于稳定时，对应的缓存大小即为工作集大小的近似值。

使用 Python 脚本 `exp2_1_workingset.py` 自动化测试。脚本遍历从 4B 到 1MB 的缓存大小，对每个大小运行模拟器，解析输出中的命中率，最后用 `matplotlib` 绘制曲线图。三个 trace 文件（`spice`、`cc`、`tex`）的指令和数据命中率分别绘制在两张子图中，便于对比分析。



4.1.1 问题 1：实验原理

本实验通过模拟不同缓存大小对命中率的影响来评估工作集大小。工作集是指在一定时间内被频繁访问的内存块集合。实验使用全相联缓存排除冲突缺失的影响，使所有缺失均为强制缺失或容量缺失，从而准确反映工作集特征。

命中率与缓存大小图表呈现三个典型阶段：

- 初始快速上升阶段表明工作集大小超过当前缓存，增加缓存可显著提高命中率；
- 逐渐平缓阶段表明接近工作集大小；
- 趋于平稳阶段表明工作集已被完全包含，进一步增加缓存不会提高命中率。

4.1.2 问题 2：大小估计

根据实验结果可估计：

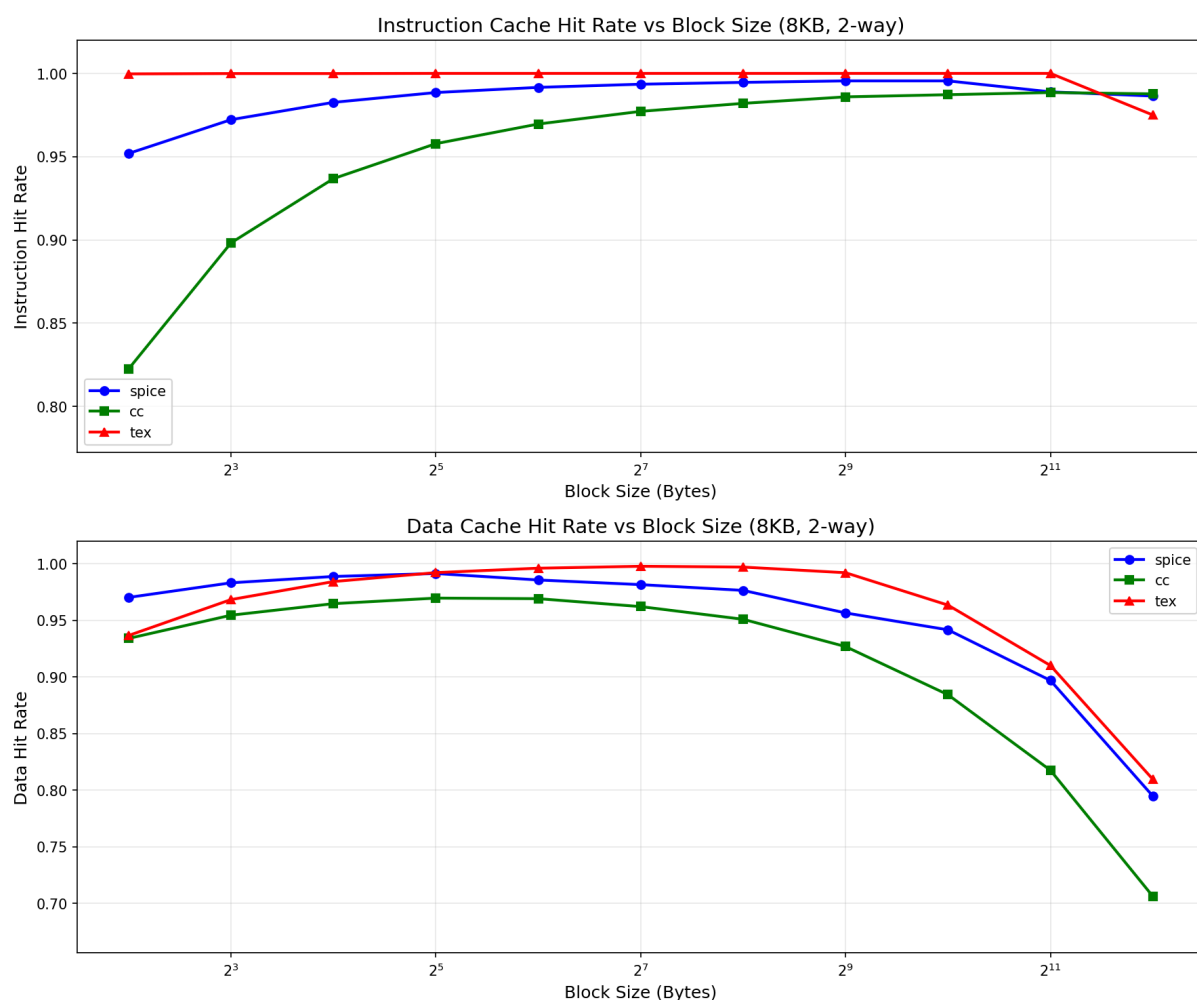
- spice.trace：指令工作集约 16KB，数据工作集约 16KB
- cc.trace：指令工作集约 32KB，数据工作集约 8KB
- tex.trace：指令工作集约 512B，数据工作集约 128B

指令工作集通常大于数据工作集，因为程序代码的访问范围较广。不同程序的工作集大小差异明显，反映了其内存访问模式的不同特征。

4.2 实验 2-2：块大小影响

本实验分析块大小对缓存性能的影响。采用分离式缓存、I-Cache 和 D-Cache 各 8KB、2 路组相联、写回+写分配策略。块大小从 4 字节变化到 4KB。较大的块大小利用空间局部性，可预取相邻数据减少缺失率；但过大的块大小会减少组数、增加冲突缺失，且增大传输开销。

使用 Python 脚本 exp2_2_blocksize.py 自动化测试。遍历 4B 到 4KB 的 11 个块大小，对每个配置运行模拟器并解析命中率，最后用 matplotlib 绘制曲线图。



4.2.1 问题 1：曲线形状

命中率随块大小呈现先上升后下降的趋势，这与空间局部性密切相关。块大小较小时，增大块可以利用空间局部性预取相邻数据，减少强制缺失，因此命中率上升。但块

大小过大时，组数减少导致冲突缺失增加，且每次缺失需传输更多数据，包括未必使用的部分，导致总体性能下降。因此曲线存在一个峰值，对应最优块大小。

4.2.2 问题 2：各 trace 最优块大小

根据实验结果，不同 trace 的最优块大小如下：

- spice.trace 指令缓存最优块大小约 512B，数据缓存约 32B；
- cc.trace 指令缓存约 2KB，数据缓存约 32B；
- tex.trace 指令缓存在 2KB 之前都基本稳定不变，直到 4KB 开始下降；数据缓存约 128B。

具体数值取决于程序的访问模式，但总体而言最优块大小在几十到几百字节范围内。

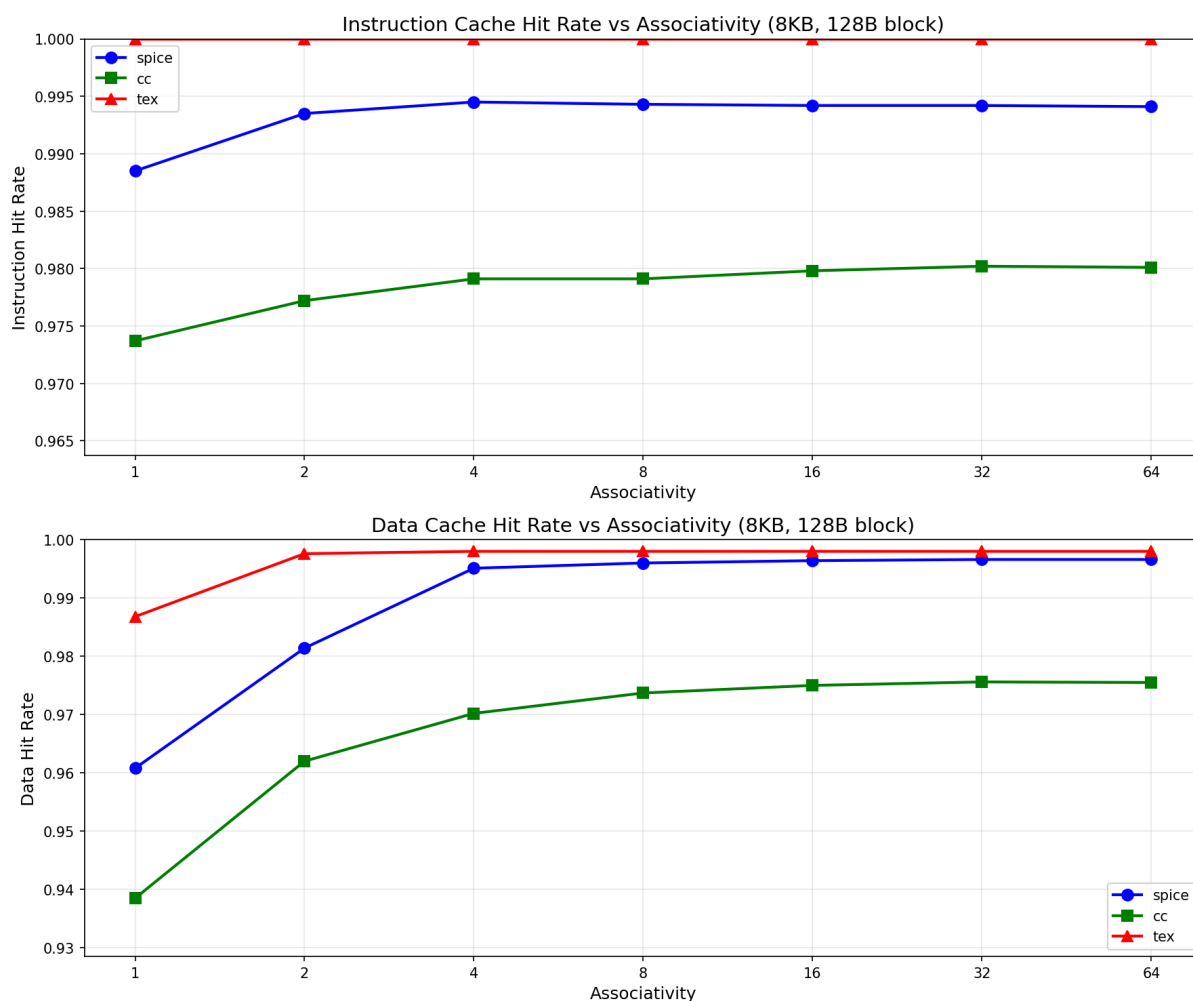
4.2.3 问题 3：差异分析

指令缓存的最优块大小通常大于数据缓存，这反映了指令和数据访问模式的本质差异。指令流主要是顺序执行，具有很强的空间局部性，较大的块可以有效预取后续指令；而数据访问模式更不规则，包括数组遍历、指针跳转、结构体访问等，空间局部性较差，过大的块会带来更多无用数据传输和冲突缺失。

4.3 实验 2-3：相联度影响

本实验分析相联度对缓存性能的影响。采用分离式缓存、I-Cache 和 D-Cache 各 8KB、128 字节块大小、写回+写分配策略。相联度从 1（直接映射）变化到 64。

使用 Python 脚本 `exp2_3_associativity.py` 自动化测试。脚本遍历相联度配置，对每个配置运行模拟器并解析命中率。纵坐标紧贴数据范围，使曲线变化更加明显。



4.3.1 问题 1：曲线形状

命中率随相联度增加而上升，但边际效益递减。这是因为增加相联度可以减少冲突缺失，当多个地址映射到同一组时，更高的相联度提供更多缓存行来容纳这些块，避免互相驱逐。

曲线初期上升较快，说明直接映射（相联度 1）时冲突缺失较严重；后期趋于平缓，说明大部分冲突已被消除，继续增加相联度收益有限。实际应用中通常采用 2-8 路组相联，在性能和硬件复杂度间取得平衡。

比较特殊的是 tex.trace 的 icache 图，命中率随着关联度几乎保持在 100%，说明该程序的指令引用具有很好的空间局部性，无论关联度如何，都能被缓存很好地覆盖。

4.3.2 问题 2：差异分析

对于指令和数据引用，命中率随着关联度的增加而上升；在达到一定关联度后，命中率的提升趋于平缓。但具体的几个阈值对于指令和数据引用可能有所不同。某些情况下，过高的关联度可能导致命中率略有下降，这在指令和数据引用中的表现可能不一致。

数据缓存从相联度增加中获益更多，这反映了指令和数据访问模式的差异。

指令流主要是顺序执行和局部跳转，地址分布较为规则，冲突缺失相对较少，因此低相联度即可获得较高命中率；而数据访问包括数组遍历、指针跳转、栈操作等，地址分布更不规则，容易产生冲突缺失，因此更能从高相联度中受益。所以现代处理器的 D-Cache 相联度通常高于 I-Cache。

4.4 实验 2-4：内存带宽分析

本实验对比不同写策略对内存流量的影响。采用分离缓存，缓存大小 8K/16K，块大小 64/128B，相联度 2/4。实验 A 比较写直通(WT)与写回(WB)策略，固定非写分配(NW)；实验 B 比较 WA 与 NW 策略，固定 WB。

使用 Python 脚本 exp2_4_bandwidth.py 自动化测试。流量统计结果如下：

实验A: Write-Through vs Write-Back (固定 Write-No-Allocate)

Trace	Config	WT+NW	WB+NW	Winner
spice	8K/64B/2way	217642	164728	WB
spice	16K/128B/4way	130858	73988	WB
cc	8K/64B/2way	557286	513682	WB

实验B: Write-Allocate vs Write-No-Allocate (固定 Write-Back)

Trace	Config	WB+WA	WB+NW	Winner
spice	8K/64B/2way	174880	164728	NW
spice	16K/128B/4way	73504	73988	WA
cc	8K/64B/2way	530816	513682	NW

4.4.1 实验 A 问题 1：WT vs WB 流量比较

WB 策略产生更少的内存流量。原因是写直通每次写操作都直接写入内存，而写回只在脏块被驱逐时才写回。当同一块被多次写入时，写回模式可将多次写操作合并为一次写回，大大减少内存流量。

4.4.2 实验 A 问题 2：WT 更优

当写操作非常稀疏且每个块只被写一次时，WT 可能更优。因为 WT 每次只写一个字，而 WB 在驱逐时需写回整个块。

在某些系统中,数据一致性要求很高,需要确保每次写操作后数据立即更新到主存。在这种情况下,WT可以减少由于数据一致性问题而导致的额外内存流量。

如果缓存替换非常频繁,WB可能会导致大量的写回操作,从而增加内存流量。而WT每次写操作都立即写回主存,避免了替换时的写回操作。

但在实际程序中,写操作通常具有时间、空间局部性,同一块多次写入很常见,因此WB通常更优。

4.4.3 实验 B 问题 1: WA vs NW 流量比较

结果取决于程序特征。WA在写缺失时需取入整个块,增加demand fetch;NW直接写内存,不增加fetch但增加copies back。当写入的数据很快被读取时,WA更优(已在缓存);当写入后不再访问时,NW更优(避免无用fetch)。

4.4.4 实验 B 问题 2: 答案反转

当程序表现出写后读模式时,WA更优,因为数据已在缓存中。当程序主要是只写模式如日志输出、结果存储时,NW更优,因为避免了不必要的块取入。实验结果显示,大多数情况下NW略优,说明写操作后续读取较少。